



操作系统课程实验

——从0到1的小型内核实现

2025年 秋季学期

lab-5 用户态虚拟内存管理 + 系统调用流程建立

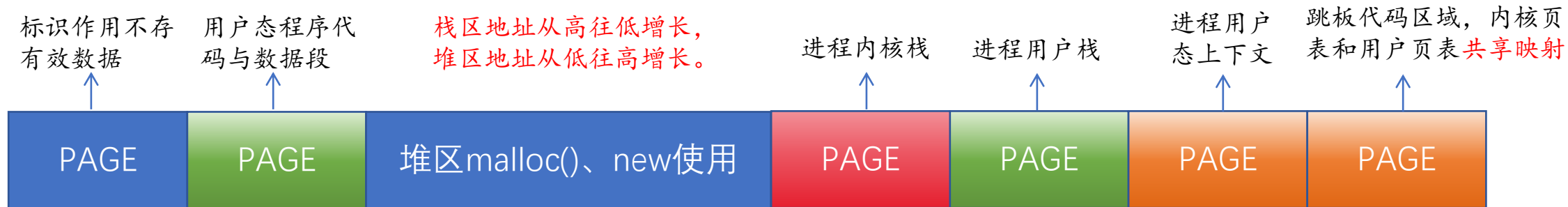
1. 用户堆区管理
2. 系统调用流程
3. 总结

1. 用户堆区管理

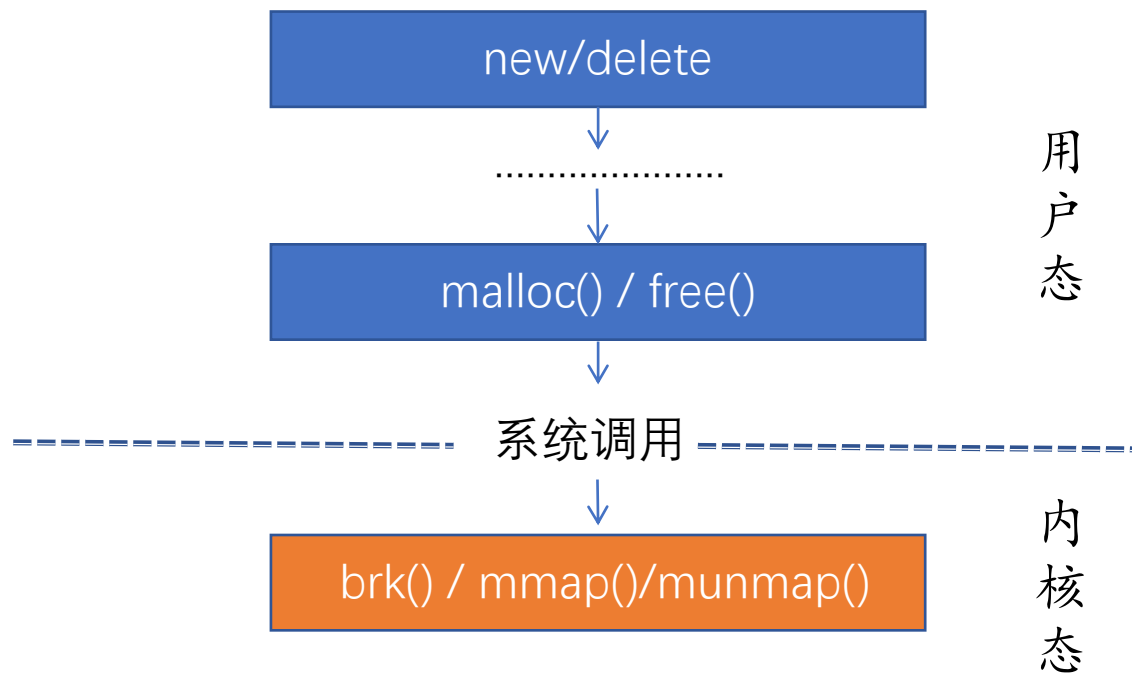
- 回顾上次实验中进程虚拟地址空间的长条模型，除了堆区之外，其他区域都有明确的功能和定义。
- 因此需要定义新的数据结构和算法来管理进程堆区。

堆区（Heap）是进程虚拟地址空间中一块由程序动态**分配和释放**的可读写内存区域。用来放“进程运行时动态分配对象”的内存区域。

用户态进程**初始状态**虚拟地址空间分布：

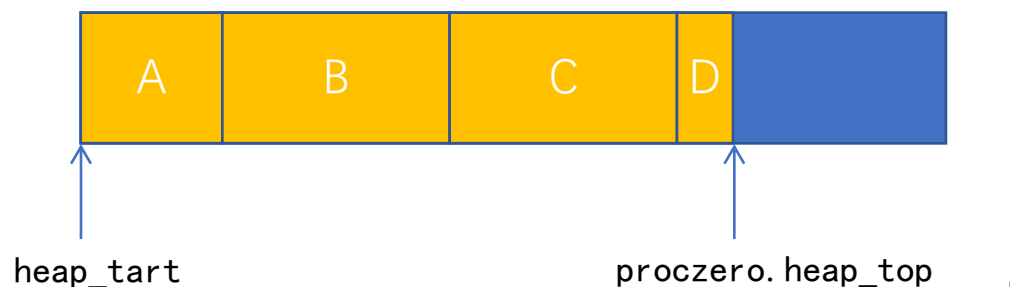


C++ 的 new / delete的函数调用路径。



1. 用户堆区管理

早期的堆区管只依靠一个“堆顶指针”（`heap_top` 或 `brk` 指针）来管理堆区，优点是快，但极易产生内存碎片。

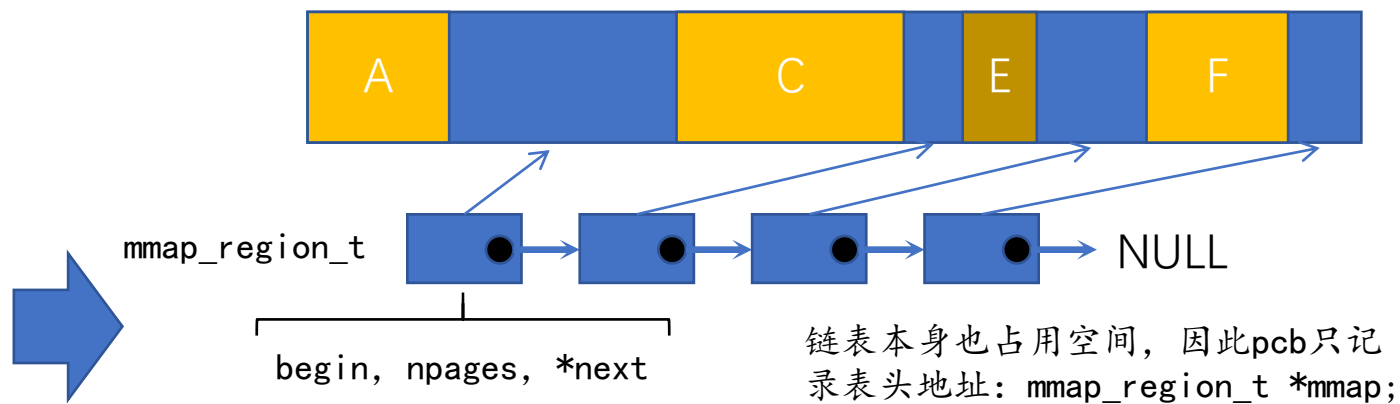


只维护了一个栈顶地址，无法记录多块地址空间堆只能像栈一样线性增长。

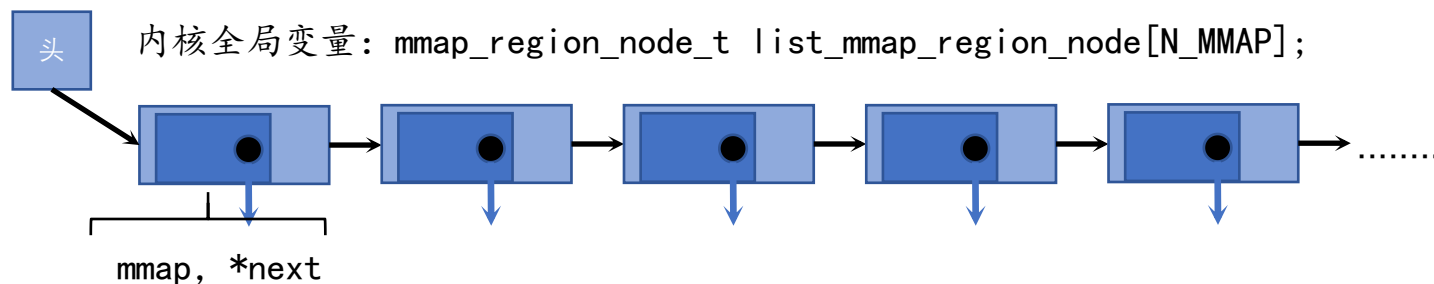


如果释放对象B，由于C没有释放，堆顶指针无法后退，B的区域无法被利用，再次分配只能向后增长

什么数据结构更适合维护分散的连续内存区域呢？毫无疑问是链表！



如果为每个进程分配单独的链表数组，会很不灵活，因此将链表节点包裹在另一个链表数组仓库（`mmap_region_node_t`）中成为内核的全局变量，这样所有进程就可以按需请求节点。



1. 用户堆区管理

堆区分配过程：遍历链表找到第一个符合条件的区域，分三种情况。



分配了mmap_region_t管理的
前半部分，只需修改
mmap_region_t中的信息



分配了mmap_region_t管理的
中间部分，申请新节点。



分配了mmap_region_t管理的全
部。回收mmap_region_t节点

堆区回收过程：链表的变化分多种情况讨论，最后统一vm_unmappages解除映射。

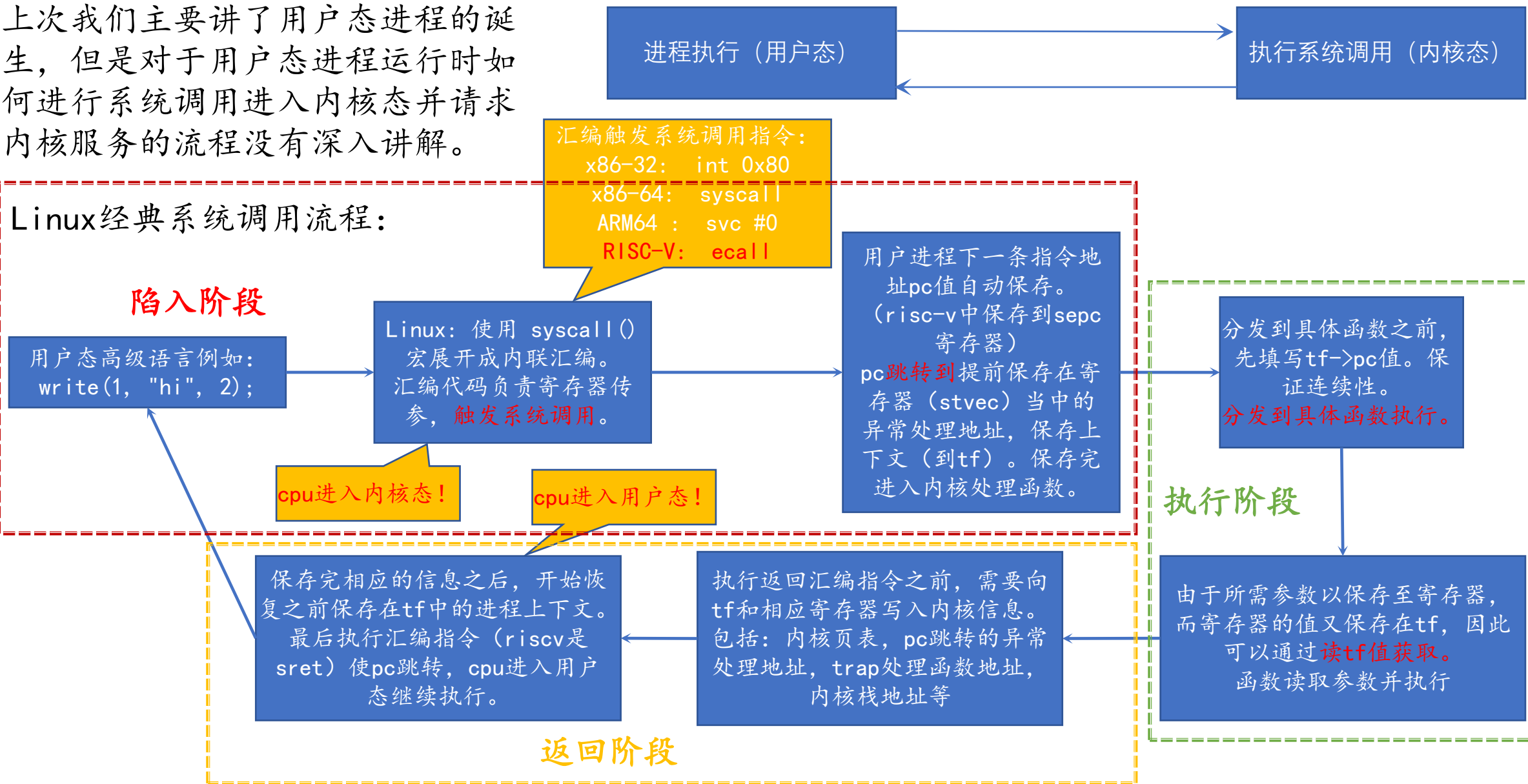


如果回收B区，则向仓库申请一个节点，重定向pcb中指针。
如果回收C区，则修改节点中的信息：起始位置，空闲页数。
如果回收D区，则合并空闲分区修改节点信息，并且仓库回收节点。



2. 系统调用流程

上次我们主要讲了用户态进程的诞生，但是对于用户态进程运行时如何进行系统调用进入内核态并请求内核服务的流程没有深入讲解。



3. 总结

- 我们本次实验介绍了用户态进程的虚拟内存（堆区）管理，堆区对于进程动态管理数据对象至关重要。
- 我们又梳理了一遍用户态进程系统调用流程的实现，与上一次只实现简单的不带参数的sys_print不同，我们展示了通用的系统调用寄存器传参方式。然而cpu中寄存器数量是有限的，因此对于大型参数比如数组，我们可以将数组地址传递给寄存器即可。
- 当我们建立了比较成熟的系统调用流程之后，我们就可以将我们在内核中实现的堆区内存管理方法包装成系统调用供用户进程使用，比如简单的堆区内内存分配和释放系统调用。
- 除此之外，同学们可以尝试在成熟的系统调用流程中实现自己的系统调用，比如一个比较成熟的可以传参的print()。